

REAL-TIME SEQUENTIAL DECISION SYSTEM FOR PRE-HOSPITAL EMERGENCY SURVIVAL

Project Technical Documentation

Document Version: 1.0 Date: October 4, 2026 Repository: F:/minor

PART 1 - EXECUTIVE SUMMARY

1.1 Problem Statement

Pre-hospital emergency response faces challenges in coordinating timely ambulance dispatch, maintaining accurate situational awareness, and ensuring protocol-driven care while preserving auditable decision trails. The system addresses the need for a modular, event-driven emergency response coordination platform.

1.2 Solution

This is a modular, event-sourced emergency response system built as a monorepo with clear module boundaries. The architecture emphasizes:

- **Deterministic safety:** Protocol logic (Module 5) is authoritative and separate from learning/analytics
- **Transaction safety:** Events are only published on successful database commits
- **Port-based architecture:** Modules communicate via interfaces, not direct dependencies
- **Auditability:** All significant state changes are persisted as immutable events

1.3 Current Implementation Status

IMPLEMENTED: Modules 1, 5, 6, 11, 12

INTERFACE/CONTRACT ONLY: Ports for Modules 7, 8, 9, 10

NOT IMPLEMENTED: Modules 2, 3, 4, 7, 8, 9, 10

1.4 Actual Verification (Repository State)

- **TypeScript:** Compiles with 0 errors (verified via `tsc --noEmit`)
- **Tests:** 169 tests passing (verified via `vitest run`)
- **Module 6 Integration:** Passing
- **Module 12 Integration:** Passing
- **Transaction Safety Tests:** 5/5 passing

1.5 Current Limitations

- No real external third-party API integrations (all are internal or simulated)
 - Modules 2-4, 7-10 not implemented
 - No production authentication/authorization beyond basic role checks
 - No real hardware GPS tracking (development/simulated)
 - No real map provider, routing provider, or traffic data
-

REAL-TIME SEQUENTIAL DECISION SYSTEM FOR PRE-HOSPITAL EMERGENCY SURVIVAL

Project Technical Documentation

Document Version: 1.0
Date: October 4, 2026
Repository: F:/minor
Prepared For: Development Team

TABLE OF CONTENTS

- 1. EXECUTIVE SUMMARY
- 2. COMPLETE SYSTEM ARCHITECTURE
- 3. MODULE-BY-MODULE STATUS
- 4. MODULE 1 - EMERGENCY CALL INTERFACE
- 5. MODULE 5 - BYSTANDER ASSISTANCE / PROTOCOL ENGINE
- 6. MODULE 6 - AMBULANCE MANAGEMENT
- 7. MODULE 11 - DATABASE & EVENT SYSTEM
- 8. MODULE 12 - DASHBOARD & LEARNING / ANALYTICS
- 9. LEARNING / ANALYTICS SAFETY BOUNDARY
- 10. APIs
- 11. EVENTS
- 12. DATABASE SCHEMA
- 13. FILE/FOLDER STRUCTURE
- 14. HOW TO RUN THE SYSTEM
- 15. HOW TO TEST
- 16. CURRENTLY WORKING
- 17. CURRENTLY SIMULATED / NOT REAL
- 18. WHAT IS NOT YET IMPLEMENTED
- 19. NEXT IMPLEMENTATION ROADMAP
- 20. REAL-WORLD HARDWARE INTEGRATION
- 21. PRODUCTION READINESS
- 22. SECURITY & SAFETY
- 23. KNOWN LIMITATIONS
- 24. FINAL PROJECT STATUS
- 25. EXACT NEXT STEPS

1. EXECUTIVE SUMMARY

1.1 Problem Statement

Pre-hospital emergency response requires rapid, coordinated decision-making under pressure. Effective response depends on timely call intake, protocol-driven guidance, appropriate ambulance dispatch, real-time tracking, and complete audit trails for all decisions.

1.2 Solution

A modular, event-driven emergency response coordination system. Core design principles:

- **Port-based architecture:** Loose coupling via interfaces (ports)
- **Transaction-safe events:** Events published only on successful commits
- **Deterministic safety:** Authoritative protocol logic separate from learning
- **Auditability:** Immutable event log for all state transitions

- **Modular boundaries:** Clear ownership per module

1.3 Current Implementation Status (Verified)

- **MODULE 1** - Emergency Call Interface: IMPLEMENTED
- **MODULE 5** - Bystander Assistance/Protocol Engine: IMPLEMENTED
- **MODULE 6** - Ambulance Management: IMPLEMENTED
- **MODULE 11** - Database & Event System: IMPLEMENTED
- **MODULE 12** - Dashboard & Learning/Analytics: IMPLEMENTED
- **MODULES 2,3,4,7,8,9,10:** NOT IMPLEMENTED / INTERFACE ONLY

Verification: TypeScript compiles with 0 errors; 169 tests passing; Module 6 and 12 integration tests pass.

1.4 External Integrations (Read-Only Audit)

No external third-party commercial APIs are integrated. All HTTP endpoints are the system's own backend REST API. Only optional local service: Ollama (localhost:11434) if running locally. Provider ports exist for future external integrations.

2. COMPLETE SYSTEM ARCHITECTURE

The system follows event-driven architecture with transaction-safe persistence. Implemented components shown as **[IMPLEMENTED]**.

ext [IMPLEMENTED] Module 1 (Call) ? (Synchronous + Events) [IMPLEMENTED] Module 5
(Protocols/Safety) ? Events/Persistence [IMPLEMENTED] Module 6 (Fleet/Dispatch) ? Module 11
(DB+Events) [IMPLEMENTED] Module 11 (Database + Event System) ? [IMPLEMENTED] Module 12
(Dashboard/Analytics)

Ports/interfaces only (future): Modules 7,8,9,10. Modules 2,3,4 not implemented.

Key architectural elements:

- **Ports** (packages/core/src/ports.ts): Contracts between modules
- **Event bus:** In-process pub/sub with transaction-scoped buffering
- **Transaction safety:** AsyncLocalStorage-based buffer; events only published on commit
- **Read-only projections:** Module 12 reads via AnalyticsReadRepository/EventQueryPort
- **Safety boundary:** Module 5 is deterministic and authoritative; learning cannot override it

3. MODULE-BY-MODULE STATUS

Module	Name	Status	Current Function	Dependencies
1	Emergency Call Interface	IMPLEMENTED	Call session lifecycle, loopback telephony, transcript append, event emission	Module 11 (repositories/ports)

2	Speech Processing	NOT IMPLEMENTED	-	Module 1
3	Emergency NLP/LLM	NOT IMPLEMENTED	-	Module 2
4	Conversation & Question Engine	NOT IMPLEMENTED	-	Module 3,5
5	Bystander Assistance/Protocol Engine	IMPLEMENTED	Deterministic protocol engine, state machine, safety checks, session management	Module 11 ProtocolSessionPort, LLM gateway (optional)
6	Ambulance Management	IMPLEMENTED	Fleet registry, assignment, dispatch, GPS tracking, nearest selection, validation	Module 11 AmbulanceRepository, EventPublisherPort
7	Hospital Intelligence	NOT IMPLEMENTED	-	Module 11, ports only
8	Route Optimization	NOT IMPLEMENTED (interfaces only)	-	RouteOptimizationPort defined
9	Emergency Corridor & Alerts	NOT IMPLEMENTED (interfaces only)	-	CorridorIntegrationPort defined
10	Autonomous Decision/State Engine	NOT IMPLEMENTED	-	Ports only
11	Database & Event System	IMPLEMENTED	Prisma ORM, migrations, repositories, transaction-safe event system, analytics	Core ports

			read projections	
12	Dashboard & Learning/Analytics	IMPLEMENTED	Live dashboard APIs, analytics aggregation, learning samples (read/write isolated), read-only projections	Module 11 AnalyticsReadRepository/EventQueryPor

Evidence from source code inspection; no external third-party APIs integrated.

4. MODULE 1 - EMERGENCY CALL INTERFACE

Status: IMPLEMENTED

Location: modules/module_01_emergency_call/

Key Files

- src/callService.ts - Core call/session lifecycle logic
- src/module.ts - Composition root
- src/api/routes.ts - HTTP routes
- src/telephony/loopbackProvider.ts - Loopback transport
- src/index.ts - Public exports

API Endpoints (from source)

- HTTP routes registered via createModule1Routes() in src/api/routes.ts

Implementation Notes

- Loopback telephony (development only); isLiveTransport false for mocks
- Session lifecycle: open, append transcript, close (idempotent)
- Events: CALL_CONNECTED, CALL_DISCONNECTED, CALL_FAILED emitted by service; database layer handles CALL_STARTED, TRANSCRIPT_UPDATED in same transaction
- Persistence via CallSessionStore, EmergencyStore (Module 11 implementations)

Tests

- ests/callService.test.ts, integration tests in ests/module01CallIntegration.test.ts (part of 169)

5. MODULE 5 - BYSTANDER ASSISTANCE / PROTOCOL ENGINE

Status: IMPLEMENTED

Location: modules/module_05_bystander_assistance/

Key Files

- src/protocol_engine/engine.ts - State machine
- src/protocol_engine/stateMachine.ts, protocolLoader.ts, Validators.ts
- src/session/sessionService.ts - Session orchestration
- src/llm/llmService.ts, ollamaProvider.ts, heuristicProvider.ts, safety.ts
- src/module.ts, src/api/routes.ts

Design (Safety Boundary)

CRITICAL: The protocol engine is the authoritative safety component. The LLM gateway provides understanding/classification assistance; it does not override deterministic protocol logic. Safety checks via `AssertSafeRephrasing()` enforce boundaries.

Protocol Catalogues

- protocols/cardiac_arrest.json, choking_adult.json, unconscious_not_breathing.json, chest_pain.json

6. MODULE 6 - AMBULANCE MANAGEMENT

Status: IMPLEMENTED

Location: modules/module_06_ambulance_management/

6.1 Key Files

- src/fleetService.ts - Registry, location, availability, assignment, dispatch
- src/assignment/assignmentService.ts - Nearest selection, scoring
- src/tracking/trackingService.ts - GPS updates, freshness
- src/api/routes.ts - REST API
- src/module.ts, src/config.ts

6.2 Ambulance Data Model

Stored in `ambulances` table (Prisma model `Ambulance`, see Module 11). Fields include: `id`, `VehicleNumber` (unique), `status`, `latitude`, `longitude`, `lastLocationUpdate`, `currentEmergencyId`, `equipmentJson`, `stationName`, `isSimulation`, etc.

6.3 How Ambulance Data Enters the System (TODAY)

Current entry points (development/simulated):

1. **API:** POST `/api/ambulances` (Module 6 routes) - creates/registers ambulance via `FleetService.registerAmbulance()` calling `AmbulanceRepository.create()`
2. **Programmatic/Tests** - Services directly
3. **Seed data** - Module 11 seed system (modules/module_11_database_event_system/src/seed/seed.ts) may create initial data if run

Initial coordinates: Provided at registration time (latitude/longitude optional). If provided, `lastLocationUpdate` set.

GPS source (current): API-driven updates only - POST `/api/ambulances/:id/location` updates position. No real hardware GPS, no external device integration. All inputs are via API calls (manual/test/simulation harness).

6.4 GPS/Location Updates

- API: POST `/api/ambulances/:id/location`
- Validation: `isValidCoordinate()` checks range (-90..90, -180..180)
- Stored in `ambulance_locations` (append-only trail) and cached on `ambulances`

- Events: AMBULANCE_LOCATION_UPDATED emitted
- Stale detection: Age calculated from `recordedAt/lastLocationUpdate`; filtered from nearest results if `> maxStaleLocationMinutes` (config, default 10) unless `includeStale=true`

6.5 Availability & Selection

- Availability determined by `Ambulance.status` (e.g. AVAILABLE, DISPATCHED, EN_ROUTE, UNAVAILABLE, etc.)
- Nearest selection: Haversine distance calculation (`haversineKm` from `packages/core/src/geo.ts`), sorted by distance, filtered by `staleness/status`
- Assignment prevents double assignment (checks active assignment), validates status
- Dispatch transitions status to EN_ROUTE and emits events

6.6 Events (Module 6)

AMBULANCE_CREATED, AMBULANCE_UPDATED, AMBULANCE_STATUS_CHANGED, AMBULANCE_LOCATION_UPDATED, AMBULANCE_ASSIGNMENT_REQUESTED, AMBULANCE_ASSIGNED, AMBULANCE_DISPATCHED, etc.

Classification: GPS/location currently **SIMULATED/DEVELOPMENT-DRIVEN** (API input only, no real hardware). Ambulance records created via API/seed in dev environment.

7. MODULE 11 - DATABASE & EVENT SYSTEM

Status: IMPLEMENTED

Location: `modules/module_11_database_event_system/`

7.1 Technology

- **Database:** SQLite (dev) via Prisma Client; schema designed for PostgreSQL compatibility
- **ORM:** Prisma
- **Transactions:** Interactive transactions with AsyncLocalStorage-scoped event buffering

7.2 Key Files

- `prisma/schema.prisma` - Schema
- `prisma/migrations/` - Migrations
- `src/db/transactionScope.ts` - Transaction-scoped event buffer (critical for safety)
- `src/events/eventService.ts` - Event persistence/publishing
- `src/events/bus.ts` - Domain event bus
- `src/repositories/*.ts` - Repository layer
- `src/module.ts` - Composition root

7.3 Transaction Safety (Critical)

- **AsyncLocalStorage** maintains per-transaction buffer (`TransactionScope.records`)
- Events buffered during transaction; only flushed via commit sink after successful commit
- On rollback, buffer cleared - no events published
- On success, buffered events published after commit
- Prevents "phantom" events for rolled-back state changes

Test coverage: `ests/transactionSafety.test.ts` (5 tests) verify rollback isolation and concurrent transaction safety.

7.4 Event System

- Append-only `system_events` table with autoincrement seq for ordering
- Event envelope includes type, emergencyId, entityId, payloadJson, metadataJson, actor, timestamp

- `recordInTransaction()` joins caller's transaction; publishes after commit
- `EventQueryPort` provides list/timeline queries

7.5 Core Models (from schema)

Emergencies, CallSessions, Transcripts, Ambulances, AmbulanceLocations (append-only), AmbulanceAssignments, Hospitals, Routes, Corridors, Notifications, SystemEvents, Decisions, Outcomes, LearningSamples, AnalyticsSnapshots, LearningModelVersions, Users, ProtocolSessions/Steps, Patients, HospitalResources.

8. MODULE 12 - DASHBOARD & LEARNING / ANALYTICS

Status: IMPLEMENTED

Location: modules/module_12_dashboard_analytics/

8.1 Components

- `src/analytics/analyticsService.ts` - Metrics, summaries, breakdowns
- `src/dashboard/dashboardService.ts` - Live state, timelines
- `src/learning/learningService.ts` - Learning samples/insights (isolated)
- `src/api/routes.ts` - Dashboard/analytics API endpoints
- `frontend/` - React dashboard UI

8.2 Data Sources

All data from Module 11 via **read-only** projections:

- `AnalyticsReadRepository` - `liveState`, `emergencyDurations`, `eventBreakdown`, `learningSamples`
- `EventQueryPort` - event timeline queries
- Real persisted data/events only; no separate domain state store

8.3 Learning Isolation (Safety Boundary)

`LearningService` reads/writes learning samples via repository but has no path to modify protocol behavior or override safety logic. See Part 9.

8.4 API Endpoints (representative)

Dashboard: `/api/dashboard/summary`, `/api/dashboard/live`, `/api/dashboard/metrics`, `/api/dashboard/ambulances`, `/api/dashboard/emergencies/:id/timeline`, `/api/dashboard/events/breakdown`

9. LEARNING / ANALYTICS SAFETY BOUNDARY

Principle: Safety-critical runtime decisions must never be influenced by learning predictions.

ext Runtime Emergency System (Protocols - Module 5) [DETERMINISTIC, AUTHORITATIVE] ? Persisted Historical Data (Module 11) ? Analytics / Learning (Module 12) ? Insights/Reports only

Never: Learning ? Override Emergency Safety Protocol

Learning layer is isolated; it cannot directly mutate protocol state or bypass safety gates. Module 5 remains the authoritative source for emergency instructions.

10. APIs (INVENTORY - ACTUAL ENDPOINTS)

Based on source inspection. All are internal backend REST APIs (served by Fastify). No external third-party API calls.

Module 6 (Ambulance Management)

- GET /api/ambulances - List ambulances
- POST /api/ambulances - Register ambulance
- GET /api/ambulances/:id - Get ambulance + latest location
- PATCH /api/ambulances/:id - Update ambulance
- POST /api/ambulances/:id/location - Update GPS location
- POST /api/ambulances/assign - Assign nearest ambulance
- POST /api/ambulances/:id/dispatch - Dispatch ambulance
- GET /api/ambulances/nearest - Find nearest available

Module 11 (Events/Emergencies)

- GET /api/events - Query event log
- GET /api/events/types - Event type vocabulary
- GET /api/events/:id - Get single event
- GET /api/emergencies/:id/events - Timeline for emergency
- GET /api/emergencies - List emergencies
- GET /api/emergencies/:id - Get emergency
- POST /api/emergencies - Create emergency
- PATCH /api/emergencies/:id - Update emergency

Module 12 (Dashboard/Analytics)

- GET /api/dashboard/summary
- GET /api/dashboard/live
- GET /api/dashboard/metrics
- GET /api/dashboard/emergencies
- GET /api/dashboard/ambulances
- GET /api/dashboard/events/breakdown
- GET /api/dashboard/emergencies/:id/timeline
- GET /api/analytics/learning/samples (admin)
- GET /api/analytics/learning/insights (admin)

Module 1 (Calls)

- Routes via createModule1Routes() (call intake endpoints)

All validation via Zod schemas in route definitions. Authentication role checks present in route metadata.

11. EVENTS CATALOGUE (ACTUAL)

Key event types (from packages/core/src/domain/events.ts, SystemEventType):

- CALL_CONNECTED, CALL_DISCONNECTED, CALL_FAILED, CALL_STARTED, TRANSCRIPT_UPDATED
- Protocol: PROTOCOL_STARTED, PROTOCOL_STEP_PRESENTED, CALLER_RESPONSE_RECEIVED, PROTOCOL_COMPLETED, PROTOCOL_ESCALATED, etc.
- Ambulance (Module 6): AMBULANCE_CREATED, AMBULANCE_UPDATED, AMBULANCE_STATUS_CHANGED, AMBULANCE_LOCATION_UPDATED, AMBULANCE_ASSIGNMENT_REQUESTED, AMBULANCE_ASSIGNED, AMBULANCE_DISPATCHED, AMBULANCE_ARRIVED_SCENE, PATIENT_ONBOARD, HOSPITAL_SELECTED, AMBULANCE_ARRIVED_HOSPITAL, EMERGENCY_COMPLETED
- Decision/Outcome/Analytics: DECISION_CREATED, OUTCOME_RECORDED, ANALYTICS_SNAPSHOT_CREATED, LEARNING_MODEL_UPDATED

All persisted in system_events with transaction-safe publication.

12. DATABASE SCHEMA (SUMMARY)

Provider: SQLite (dev), compatible with PostgreSQL. Tables include: users, emergencies, call_sessions, transcripts, protocol_sessions/steps, ambulances, ambulance_crew, ambulance_locations (append-only), ambulance_assignments, hospitals, routes, route_updates, corridors, hospital_resources, patients, notifications, system_events, decisions, outcomes, learning_samples, analytics_snapshots, learning_model_versions.

Key constraints: foreign keys, unique constraints (e.g. vehicleNumber), indexes on status/timestamps. Event log is append-only by design.

13. FILE/FOLDER STRUCTURE (KEY PATHS)

- packages/core/ - Shared types, ports, domain models, events
 - modules/module_01_emergency_call/ - Call interface (IMPLEMENTED)
 - modules/module_05_bystander_assistance/ - Protocol engine (IMPLEMENTED)
 - modules/module_06_ambulance_management/ - Fleet management (IMPLEMENTED)
 - modules/module_11_database_event_system/ - DB+Events (IMPLEMENTED)
 - modules/module_12_dashboard_analytics/ - Dashboard/Analytics (IMPLEMENTED)
 - Ⓜpps/server/, Ⓜpps/dashboard/ - Application hosts
 - docs/ - Documentation
-

14. HOW TO RUN THE SYSTEM

Prerequisites: Node.js >= 20, npm

Setup: `Ⓜash npm install npm run db:generate npm run db:migrate npm run db:seed # if available`

Development: `Ⓜash npm run dev # runs server + web concurrently`

Build/Test: `Ⓜash npm run build npm run typecheck npm test`

Commands from package.json. Database uses SQLite by default (DATABASE_URL).

15. HOW TO TEST

Actual test results (verified): 169 tests passing, 0 failed. TypeScript 0 errors.

Commands: `Ⓜash npm test # Run all tests npm run typecheck`

Test breakdown: Unit and integration tests across modules, including transaction safety tests (5/5 passing), Module 6/12 integration tests passing.

16. CURRENTLY WORKING (VERIFIED)

- ? Call session lifecycle (Module 1)
- ? Protocol engine with state machine (Module 5)
- ? Ambulance registration/management (Module 6)
- ? Nearest ambulance selection (haversine)
- ? Assignment with concurrency checks (Module 6)
- ? Dispatch lifecycle (Module 6)
- ? GPS location updates with validation (Module 6)

- ? Stale location filtering (Module 6)
- ? Transaction-safe event persistence (Module 11)
- ? Event log with ordering and audit (Module 11)
- ? Learning sample persistence with featuresJson (Module 11)
- ? Dashboard APIs and services (Module 12)
- ? Analytics from persisted data (Module 12)
- ? Learning isolated from safety logic (Module 12)

17. CURRENTLY SIMULATED / NOT REAL

Component	Status	Notes
Ambulance GPS	SIMULATED/INPUT-DRIVEN	API input only (POST /api/ambulances/:id/location). No real hardware, no phone GPS, no external tracking devices.
Routing/Traffic	SIMULATED	Great-circle distance + estimateTravelMinutes; rafficFactor configurable, not real traffic data.
Map Provider	NOT CONFIGURED/NOT REAL	No external map API integrated.
Hospital Data	DEVELOPMENT DATA	Database records only; no external hospital system integration.
Emergency Corridor	NOT IMPLEMENTED	Interfaces only, no real traffic signal integration.
Notifications (SMS/Push)	NOT IMPLEMENTED	Models only; no external providers.
LLM (external)	LOCAL-OPTIONAL	Uses Ollama on localhost if available; heuristic fallback exists. No commercial cloud LLM APIs integrated.
Speech/STT/TTS	SIMULATED/MOCK	Provider ports defined; loopback implementation. No external speech providers.
Telephony	SIMULATED (LOOPBACK)	Loopback provider for development; not live telephony.

External third-party APIs: None integrated (read-only audit confirms).

18. WHAT IS NOT YET IMPLEMENTED

Module	Status	Key Missing Work
2 - Speech Processing	NOT IMPLEMENTED	STT/TTS implementations via provider ports
3 - Emergency NLP/LLM	NOT IMPLEMENTED	Fact extraction, classification logic

4 - Conversation & Question Engine	NOT IMPLEMENTED	Dialog management, question selection
7 - Hospital Intelligence	NOT IMPLEMENTED	Capacity, bed availability, diversion, suitability scoring
8 - Route Optimization	INTERFACE ONLY	Real routing provider integration (no external APIs wired)
9 - Emergency Corridor & Alerts	INTERFACE ONLY	Corridor negotiation, geofencing, notifications to road users
10 - Autonomous Decision/State Engine	NOT IMPLEMENTED	System orchestration, state coordination

19. NEXT IMPLEMENTATION ROADMAP (RECOMMENDED)

1. **Module 7 - Hospital Intelligence** (core dependency for hospital selection)
2. **Module 8 - Route Optimization** (leverages Module 6 fleet + 7 hospital)
3. **Module 9 - Emergency Corridor & Alerts** (depends on routes)
4. **Module 3 & 4** (speech understanding + conversation)
5. **Module 2** (speech I/O)
6. **Module 10** (orchestration)
7. **Integration testing across full flow**
8. **Hardware/GPS integration (if moving to real deployment)**

Rationale: Build data/selection layers (7,8) before real-time coordination (9,10) and I/O layers (2,3,4).

20. REAL-WORLD HARDWARE INTEGRATION

Ambulance GPS tracking path: GPS Hardware (GPS module) ? Microcontroller (ESP32/RPi) ? Cellular (4G/LTE) ? Backend API (HTTPS) ? Module 6 ? Module 11 ? Module 12

Interface requirements for hardware client:

- Authenticated HTTPS POST to /api/ambulances/:id/location
- Payload: latitude, longitude, optional speedKmh, headingDeg, AccuracyM, imestamp
- Update frequency: configurable (e.g. 1-5s moving, 30s idle)
- Offline buffering + retry on reconnect
- Device identification (ambulancelid/vehicle)
- Timestamp sync (use server time or include recordedAt)
- TLS, auth token/credentials

Notes: No specific hardware assumed; system is ready to accept API inputs from any compliant GPS tracker/app.

21. PRODUCTION READINESS

Current classification: READY FOR DEVELOPMENT / DEMO

NOT PRODUCTION READY for real emergency deployment

Gaps: Authentication/authorization hardening, external provider integrations (if required), monitoring/logging/alerting, deployment infra, real-time scaling, failover, backup/recovery, PII/privacy review,

compliance/safety validation.

22. SECURITY & SAFETY

- Input validation via Zod
- Role-based auth checks in route metadata
- Transaction integrity for critical state changes
- Append-only audit trail (events)
- Safety boundary: Module 5 deterministic; learning isolated
- No external credentials required in current codebase

23. KNOWN LIMITATIONS

Limitation	Impact	Workaround	Future Work
No real GPS hardware	Development only	API/manual updates	Hardware integration
No external routing/maps	Distance estimates only	Haversine estimates	Integrate routing provider
Modules 2-4,7-10 incomplete	Partial end-to-end flow	Focus on implemented modules	Implement remaining modules
No production auth	Demo use only	Development config	Add proper authN/authZ

24. FINAL PROJECT STATUS

Module	Status	Tested	Integration Ready
1	IMPLEMENTED	Yes	Yes
2	NOT IMPLEMENTED	No	No
3	NOT IMPLEMENTED	No	No
4	NOT IMPLEMENTED	No	No
5	IMPLEMENTED	Yes	Yes
6	IMPLEMENTED	Yes	Yes
7	NOT IMPLEMENTED	No	No
8	INTERFACE ONLY	No	No
9	INTERFACE ONLY	No	No
10	NOT IMPLEMENTED	No	No
11	IMPLEMENTED	Yes	Yes
12	IMPLEMENTED	Yes	Yes

Current Completion: ~42% (5 of 12 modules fully implemented; 2 interface-only)

TEST STATUS: 169 passed / 0 failed

TYPECHECK: 0 errors

BUILD: Configured (via package.json scripts)

EXTERNAL INTEGRATIONS: None (all internal APIs; Ollama local-optional)

25. EXACT NEXT STEPS

A. Software Implementation

1. **Module 7 (Hospital Intelligence)** - Implement hospital capacity/resource tracking, suitability scoring. *Why:* Required for intelligent hospital selection.
2. **Module 8 (Route Optimization)** - Implement routing logic behind RouteOptimizationPort (can start with deterministic logic; external APIs optional). *Why:* Needed for accurate ETAs.
3. **Module 3 (NLP/LLM) + Module 4 (Conversation) + Module 2 (Speech)** in logical order. *Why:* Complete call?understand?interact flow.

B. Hardware Integration

1. Define API contract for GPS devices (already exists).
2. Prototype with test client sending location updates.

C. External Services

- None required for core functionality. Can add optional providers later via ports.

D. Testing

- Continue integration tests as modules added; maintain existing 169 passing.

E. Final Demonstration

- Demo with implemented modules (1,5,6,11,12) showing registration?assignment?dispatch?tracking? dashboard.

F. Deployment

- Not recommended for production until auth, monitoring, and required modules complete.

This documentation reflects the ACTUAL repository state as of October 4, 2026. All technical claims verified against source code. External API audit confirms no third-party commercial integrations are present.